PMS Hotelero · Piloto — Especificaciones técnicas

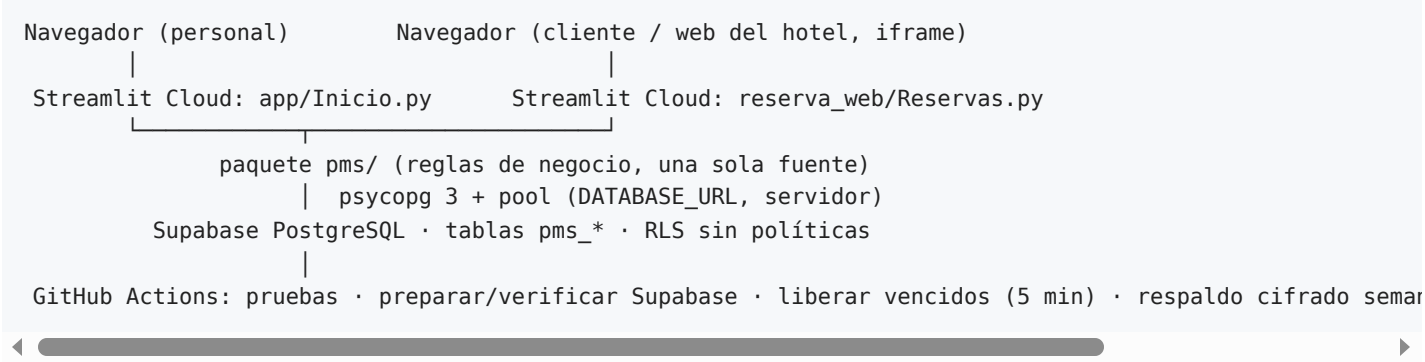
Prototipo diseñado para ATm Consulting · versión 1.0 · 29 de octubre de 2026 · todos los datos son ficticios

1. Resumen

El PMS Hotelero Piloto es un sistema de gestión hotelera multi-país (Colombia/COP, República Dominicana/DOP y un tercer país genérico) con impuestos configurables, 4 roles, reservas sin doble venta, check-in/check-out, housekeeping, bloqueos, facturación simple con comprobante PDF (no es factura legal), reportes, auditoría inmodificable, **plano de habitaciones** y una **mini web pública de reservas** con pasarela y correo simulados.

Elemento	Valor
App interna	https://pms-hoteles-atm.streamlit.app/ (`app/Inicio.py`)
Mini web de reservas	https://pms-reservas-atm.streamlit.app/ (`reserva_web/Reservas.py`)
Código	https://github.com/Fredchars/pms-hoteles (público, rama `main`)
Base de datos	Supabase Free, proyecto *atlas-selleragent*, 21 tablas con prefijo `pms_` y RLS
Pruebas	133 pruebas automáticas (pytest) en 16 archivos
Datos semilla	3 hoteles × 18 habitaciones (3 plantas × 6), 19 usuarios de prueba

2. Arquitectura



- **Interfaz:** Streamlit 1.66 (Python 3.13), fondo blanco, marca ATm y aviso permanente de datos ficticios.
- **Lógica:** paquete `pms/` compartido por ambas apps; la mini web solo importa `pms/reserva_web.py` .
- **Datos:** PostgreSQL (Supabase). La API REST pública de Supabase no ve nada (RLS activo y permisos revocados a `anon` / `authenticated`).

- **Dependencias:** streamlit 1.66.0, psycopg[binary] 3.3.6, psycopg-pool 3.3.3, pandas 3.0.6, bcrypt 4.2.0, reportlab 5.1.0.

3. Módulos del código

Módulo	Responsabilidad
`pms/db.py`	Conexión a PostgreSQL con pool pequeño
`pms/seguridad.py`	Roles, permisos por pantalla, contraseñas bcrypt, sesión firmada (8 h), bloqueo por 5 fallos en 15 min, primer ingreso y contraseña temporal
`pms/config.py`	Hoteles (país, moneda, zona horaria), impuestos, tipos, habitaciones, camas, usuarios
`pms/disponibilidad.py`	Habitaciones libres por noche y capacidad
`pms/tarifas.py`	Tarifas por tipo y temporada, estancia mínima, cierre de ventas, cotización
`pms/reservas.py`	Crear, modificar, cancelar (aprobación de gerencia), no-show, check-in, check-out
`pms/vencimientos.py`	Vencimiento de opciones (48 h recepción, 15 min web)
`pms/housekeeping.py`	Estados de limpieza y transiciones válidas
`pms/bloqueos.py`	Bloqueos por mantenimiento
`pms/facturacion.py`	Cargos, pagos, descuentos con aprobación, impuestos congelados, comprobante PDF
`pms/huespedes.py`	Huéspedes, consentimientos (Ley 1581), anonimización
`pms/plano.py`, `pms/dibujo.py`	Estado de cada habitación para el plano y dibujo HTML de cada planta
`pms/reserva_web.py`, `pms/pasarela.py`, `pms/correo.py`	Reserva web, pasarela simulada (HMAC) / esqueleto Wompi, correo simulado / Resend-Brevo
`pms/auditoria.py`, `pms/vistas.py`	Bitácora de solo agregar; consultas de dashboard, calendario y reportes
`app/paginas/*.py`	13 páginas: dashboard, calendario, plano, reservas, huéspedes, housekeeping, bloqueos, tarifas, facturación, reportes, reservas web, configuración, auditoría

4. Modelo de datos

21 tablas `pms_*`, todas (salvo las bitácoras globales) con `hotel_id`. Detalle completo, diagramas ER y diccionario en el **Anexo A · Modelo de datos**. Reglas clave en la base: - Índice único parcial `ux_pms_noche_habitacion (habitacion_id, noche) where activa` → **imposible vender dos veces una**

habitación, incluso con dos recepcionistas a la vez. - Precio por noche congelado en `pms_reserva_noches.precio`; impuestos congelados en `pms_cargos.impuestos` (jsonb). - `pms_auditoria` protegida por un trigger que impide `update` y `delete`.

5. Estados y transiciones

Reserva: `consulta` → `opcion` → `confirmada` → `alojado` → `salida`; alternativas `cancelada` (requiere aprobación de gerencia; quien pide no aprueba) y `no_show`. Las opciones nuevas llevan `opcion_vence_en` y se liberan solas.

Housekeeping: `sucia` → `en_limpieza` → `inspeccion` → `lista`; cualquiera puede pasar a `fuera_de_servicio`, que vuelve a `sucia`.

Reserva web: `bloqueada` (15 min) → `pago_pendiente` → `pagada` → `confirmada`; alternativas `fallida`, `expirada`, `cancelada`. Pago por pasarela: `pendiente`, `aprobada`, `rechazada`, `vencida`, `requiere_revision`.

6. Flujos principales

1. **Reserva en recepción:** huésped (con consentimiento) → fechas, personas, tipo → desglose por noche con precio e impuestos → guardar (asigna habitación y crea noches).
2. **Desde el plano:** elegir fechas y habitación en el plano de la planta → «Continuar con la reserva» abre Reservas prellenado.
3. **Check-in / check-out:** crea `pms_estancias`; al salir la habitación pasa a `sucia`. Gerencia puede forzar salida con saldo.
4. **Cuenta:** cargos (noche, extra, descuento pendiente de aprobación) y pagos (efectivo, datáfono, transferencia, otro, pasarela) → comprobante PDF.
5. **Reserva web:** buscar → bloquear 15 min → datos + consentimiento → pago simulado firmado → número `PMS-AAAA-XXXXXX` y correo simulado → aparece en el PMS con canal `web`.

7. Plano de habitaciones

Página «Plano de habitaciones» (recepción y gerencia la usan; administrador solo consulta; housekeeping no la ve). Muestra una planta a la vez con las 6 habitaciones del layout provisional (x01 Familiar Mar, x02 Triple Mar, x03/x04 Doble Monte, x05 Doble Patio, x06 Individual), colores por estado (disponible, ocupada, en opción, bloqueada, fuera de servicio, seleccionada, capacidad insuficiente), filtros (fechas, adultos, niños, tipo, solo vista al mar, ocultar no disponibles) y detalle con camas y precio. Usa la misma disponibilidad y tarifas que Reservas. La vista al mar es la columna `pms_habitaciones.vista_mar`.

Cada planta se dibuja como plano arquitectónico (`pms/dibujo.py`, HTML puro con fondo blanco, compartido por el PMS y la mini web): franja del mar arriba, habitaciones con vista al mar en la fila del mar y el resto al otro lado del pasillo central, zonas comunes como rótulos, camas dibujadas a escala dentro de cada habitación, metros cuadrados, rótulo **VISTA MAR** y color de estado.



8. Seguridad

- RLS activado en todas las tablas `pms_*` sin políticas y permisos revocados a los roles públicos (`db/02_rls_pms.sql`). Solo el servidor accede con `DATABASE_URL`.
- Contraseñas con bcrypt; primer ingreso obliga a crear contraseña propia (mín. 10 caracteres); el administrador genera temporales que se muestran una sola vez.
- Sesión firmada con `PMS_SESION_SECRETO`, 8 h; cambiar la clave invalida sesiones anteriores.
- `PMS_ENTORNO=produccion` apaga siempre el acceso de demostración.
- Reserva web: token de enlace HMAC (solo se guarda su hash), límites por IP y correo (hasheados), validación de entradas, eventos de pago firmados e idempotentes.
- Permisos por pantalla (x = usa y modifica · ver = consulta):

Pantalla	Recepción	Gerencia	Housekeeping	Administrador
Dashboard, Calendario, Plano, Reservas, Huéspedes, Facturación	x	x	—	ver
Housekeeping	ver	x	x	ver
Bloqueos	ver	x	ver	x
Tarifas	ver	x	—	x

Pantalla	Recepción	Gerencia	Housekeeping	Administrador
Reportes	—	x	—	x
Configuración	—	ver	—	x
Auditoría	—	ver	—	ver

9. Tareas programadas (GitHub Actions)

Workflow	Cuándo	Qué hace
Pruebas automáticas (`pruebas.yml`)	cada push	133 pruebas con PostgreSQL de servicio
Preparar y verificar Supabase (`supabase.yml`)	manual	Crea/migra tablas `pms_*`, RLS, semilla, marca cambio de clave y verifica que la API pública no devuelve nada
Liberar vencidos (`liberar-vencidos.yml`)	cada 5 min	Libera opciones y bloqueos web vencidos
Respaldo semanal (`respaldo.yml`)	domingos 3:45 a. m. (Colombia)	`pg_dump` de `pms_*` cifrado AES-256, artefacto 30 días
Verificar reserva web (`verificar-reserva-web.yml`)	manual	Reserva de punta a punta con datos ficticios y limpieza

10. Pruebas automáticas

133 pruebas en `tests/`: doble venta y concurrencia, precio congelado, impuestos, permisos por rol, estados, aprobaciones, seguridad de datos, primer ingreso, plano (lógica e interfaz), reserva web (34) e interfaz de reserva web (9). Se niegan a correr contra un servidor que no sea local.

11. Implementado frente al diseño aprobado

Requisito del diseño	Estado
Multi-hotel, multi-moneda, impuestos genéricos	✅
4 roles con permisos por pantalla	✅
Reservas sin doble venta, precio congelado	✅
Check-in/out, housekeeping, bloqueos	✅
Facturación simple y comprobante PDF	✅ (no es factura electrónica DIAN)

Requisito del diseño	Estado
Consentimiento Ley 1581 y anonimización	✅
Auditoría inmodificable	✅
Plano de habitaciones (ampliación aprobada)	✅ (plano dibujado por planta)
Reserva web con pasarela y correo	✅ prototipo simulado
Channel manager, factura electrónica, pasarela real	Fases siguientes

12. Limitaciones conocidas

- Streamlit Community Cloud duerme la app tras horas sin visitas y no recibe webhooks (la pasarela real necesita un receptor aparte).
- Supabase Free puede pausarse por inactividad; el respaldo semanal ayuda a mantenerlo activo.
- Las tareas de GitHub Actions pueden retrasarse y se pausan tras 60 días sin actividad del repositorio.
- La mini web usa la misma credencial de base que el PMS; en producción, crear un rol con permisos mínimos.
- Una reserva web = una habitación por toda la estancia. Sin captcha comercial.
- Datos ficticios: no cargar datos reales de huéspedes en el piloto.

13. Configuración: variables

Variable	App	Para qué
`DATABASE_URL`	ambas	Conexión a Supabase (solo en secretos)
`PMS_SESION_SECRETO`	ambas	Firma de sesión / enlaces
`PMS_MODO_DEMO`	PMS	`no` en producción
`PMS_ENTORNO`	ambas	`produccion` apaga la demo
`PMS_PASARELA_SECRETO`, `PMS_WEB_URL`	web	Firma del webhook simulado, URL pública
`PMS_WEB_HOLD_MINUTOS`, `PMS_HORAS_OPCION`	web	15 y 48 por defecto
`PMS_CORREO_*`	web	Opcional: correo real con lista de destinatarios permitidos

Los valores reales **no** están en el repositorio ni en este documento; ver la *Guía de continuidad y respaldo*, sección 2.